

option info MP

Correction DS3

1)

2)

3)

4)

Soit G un graphe connexe. Soit a une arrête appartenant à un cycle., et $x, y \in S$. Soit $x_0, \dots, x_n, y$ un chemin de x à y .

- Soit ce chemin ne passe pas par a , et donc il est encore dans $G - a$.
- Soit ce chemin passe par a , et a relie x_i et x_{i+1} . Alors, on pose $x_i, x_{i+1}, z_1, \dots, z_k, x_i$ le cycle qui contient a . Le chemin $x_1, \dots, x_i, z_k, \dots, z_1, x_{i+1}, \dots, x_n$ est un chemin de x à y qui ne passe pas par a , et donc il est dans $G - a$

Donc $G - a$ est connexe.

5)

Soit $G = (S, A)$, connexe, tq $|A| = |S| - 1$.

Alors, si G avait un cycle, on peut prendre a une arrête de ce cycle. D'après la question précédente, $G - a$ est connexe. Or, $G - a$ possède $|S| - 2 < |S| - 1$ arrêtes. D'après la proposition 1, c'est absurde.

Donc G est acyclique, et, comme il est connexe, c'est un arbre.

6)

Soit G un graphe connexe, avec n sommets et $n - 1 + p$ arrêtes.

$p = 0$ alors G est un arbre d'après la question 5, ok

$p > 0$ alors G a un cycle par la proposition 1, et donc avec a une arrête de ce cycle, $G - a$ est un graphe connexe avec n sommets et $n + p - 2$ arrêtes. Par hypothèse de récurrence, $G - a$ admet un arbre couvrant, et G aussi.

Inversement, si G admet un arbre couvrant, c'est qu'il admet un sous-graphe connexe, et donc il est connexe (tout chemin dans l'arbre couvrant en est aussi un dans G).

7)

7.a)

$T + a$ possède $|S|$ arrêtes, donc il n'est pas acyclique d'après la proposition 1.

7.b)

$T + a$ est connexe, et a' est une arrête d'un de ses cycles. Donc, d'après la question 4, $T + a - a'$ est connexe, et $|A| = |S| - 1 + 1 - 1 = |S| - 1$. D'après la question 5, $T + a - a'$ est un arbre, et c'est un sous-graphe de G , donc s'en est un arbre couvrant.

8)

D'après la question 6, G admet un arbre couvrant. De plus, étant donné que G est fini, il admet un nombre fini d'arbres couvrants distincts. Donc, il en existe un (ou plusieurs) de poids maximal.

9)

9.a)

$T_1 + a_2$ possède un cycle, car il contient $|S|$ arrêtes. Or, si toutes les arrêtes de ce cycle étaient dans A_2 , alors T_2 contiendrait un cycle. Donc au moins une de ces arrêtes n'est pas dans A_2 , et on a donc $a_1 \in A_1$.

A_2 .

9.b)

D'après la question 7b, $T_1 + a_2 - a_1$ est un arbre couvrant. De plus, comme a_2 est de poids maximal parmi les arrêtes de $(A_1 \setminus A_2) \cup (A_2 \setminus A_1)$, et que $a_1 \in A_1 \setminus A_2$, on a forcément $p(a_1) < p(a_2)$.

Donc $P(T_1 + a_2 - a_1) > P(T_1)$. Absurde, car T_1 est de poids maximal !

10)

```
let max_tab t =  
  let n = Array.length t in  
  let rec aux i = if i = n-1 then t.(i) else  
    max t.(i) (aux (i+1))  
  in  
  aux 0;;
```

```
let max_tab t =  
  let m = ref t.(0) in  
  for i = 1 to (Array.length t) - 1 do  
    m := max (!m) t.(i);  
  done;  
  !m;;
```

11)

```
g.(7) = [(8, 19.0); (0, 18.0)]
```

12)

```
let ajoute_arete g (w, i, j) =  
  g.(i) <- (j, w)::(g.(i));  
  g.(j) <- (i, w)::(g.(j));;
```

13)

```
let toutes_les_aretes g =  
  let n = Array.length g in  
  let rec parcours_tab i =  
    if i >= n then []  
    else  
      let rec parcours_list adj = match adj with  
        | [] -> parcours_tab (i+1)  
        | (j, w)::q when i < j -> (w, i, j)::(parcours_list q)  
        | (j, w)::q -> parcours_list q
```

```

        in
        parcours_list g.(i)
    in
    parcours_tab 0;;

```

Dans chaque appel à `parcours_tab i`, on appelle une fois `parcours_list` pour chaque voisin de `i`, puis on appelle `parcours_tab i + 1`.

Au total, on a donc 2 appels à `parcours_list` par arrête, et un appel à `parcours_tab` par sommet.

Soit un $O(|S| + |A|)$

14)

```

let rec min_arete l = match l with
| [(w, a, b)] -> (w, a, b)
| (w, a, b)::q = let (wmin, amin, bmin) = min_arete q in
    if w < wmin then (w, a, b) else (wmin, amin, bmin)
;;

```

15)

Elle explore le graphe en partant d'un sommet, et note tous les sommets rencontrés à `k`.

16)

```

let composantes_connexes g =
    let n = Array.length g in
    let cc = Array.make n (-1) in
    let k = ref 0 in

    for i = 0 to n - 1 do
        if cc.(i) < 0 then begin
            explorer g cc (!k) i;
            incr k
        end
    done

    cc;;

```

17)

```

let est_connexe g =
    let cc = composantes_connexes g in
    let connexe = ref true in
    let n = Array.length g in
    for i = 0 to n-1 do
        connexe := (!connexe) && cc.(i) = 0
    done;
    !connexe

```